

@AI Engineer

RAG

Interview Question & Answers



DEVENDHIRAN
AI Engineer

Q1. Explain the requirement of RAG when LLMs are already powerful.

LLMs are powerful, as they are trained on large volumes of data using sophisticated techniques. However, LLMs because of knowledge cutoff (static knowledge), struggle to answer queries related to the latest events or the data not present in their training corpus.

RAG addresses this challenge by retrieving relevant context from external knowledge sources, which allows LLMs to provide accurate responses. This is why RAG is essential for LLM-based applications that need to be accurate. Otherwise, LLMs alone might provide you answers that are incomplete or outdated.

Q2. Is RAG still relevant in the era of long context LLMs?

RAG is still important even with long context LLMs. This is because long-context LLMs without RAG have three big problems: "lost in the middle," high API costs, and increased latency.

Long-context LLMs often struggle to find the most relevant information in large contexts, which hurts the quality of generated responses. Furthermore, processing lengthy sequences in each API call results in high latency and high API costs.

RAG addresses these issues by providing the most relevant information from external knowledge sources. So, you still need RAG to get accurate and cost-efficient responses, even with long context LLMs.

Q3. What are the fundamental challenges of RAG systems?

RAG is powerful, but it has to deal with the following challenges:

Scalability: Searching and retrieving from large, dynamic knowledge sources quickly and efficiently requires a lot of computing power and well-optimized indexing, which can be expensive or take a long time.

Latency - The two-step process (retrieval then generation) can cause delays, making it less suitable for real-time applications without careful optimization.

Hallucination Risk - Even with retrieval, the model might generate plausible but unsupported details if the retrieved data is ambiguous or insufficient.

Q4. What are effective strategies to reduce latency in RAG systems?

Caching, embedding quantization, selective query rewriting, and selective re-ranking are

some of the ways to reduce RAG latency. Caching stores retrieved results or generated responses to avoid redundant computation. Embedding quantization to lower bit precision reduces memory and computational load, speeding up retrieval.

Selective query rewriting enhances recall and relevance by refining queries prior to retrieval, primarily utilized for complex or ambiguous queries. Selective re-ranking is only used for complicated queries, which cuts down on unnecessary computation for simpler ones.

Q5. Explain R, A, and G in RAG.

RAG stands for Retrieval-Augmented Generation. The "R" or Retrieval, refers to the process of searching and fetching the most relevant information from external knowledge sources for the given user query.

The "A" or Augmented, involves including the retrieved relevant context in the LLM prompt having the user query and instructions so that the LLM can generate a response based on the provided context.

Finally, the "G" or Generation is the phase during which the generator LLM processes the prompt having instructions, a query, and context to generate a response that is coherent, accurate, and contextually relevant.

Q6. How does RAG help reduce hallucinations in LLM generated responses?

Without RAG, LLM answers user questions based on what it learned from the training corpus, which may not be up-to-date or complete. This could lead to hallucinated responses, which are answers that sound right but are wrong.

Retrieval-Augmented Generation (RAG) helps cut down on hallucinations in LLM-generated responses by adding an external retrieval system that pulls relevant, factual information from trusted, up-to-date external knowledge sources.

By combining retrieval with generation, RAG ensures that answers are more accurate, contextually relevant, and less prone to fabrications or false information, significantly enhancing the reliability of the output.

Q7. Why is re-ranking important in the RAG pipeline after initial document retrieval?

The top K chunks fetched by the RAG retriever may have irrelevant chunks ahead of relevant ones. Passing these results directly to the LLM hurts the quality of the answers

because LLMs mostly look at the top-ranked chunks that are given as context.

Re-ranking uses cross-encoder models to deeply measure the semantic relevance of query-chunk pairs and then brings relevant chunks ahead of irrelevant chunks. This reduces the noise and helps the generator LLM to generate more accurate and coherent answers.

Q8. What is the purpose of character overlap during chunking in a RAG pipeline?

In a RAG pipeline, chunk overlap during chunking ensures contextual continuity and

prevents loss of information at the boundaries of chunks. This improves the retrieval

accuracy and maintains coherence in the text fed to the LLM.

Typically, an overlap of about 10-20% of the chunk size is used to strike a balance between preserving context and computational efficiency in RAG applications.

Q9. What role does cosine similarity play in relevant chunk retrieval within a RAG pipeline?

Cosine similarity measures how similar the query embedding is to the embeddings of chunks in the vector database. It finds the cosine of the angle between two vectors and provides a score that shows how closely related the query is to each chunk. Higher scores mean that the chunk is more relevant.

This enables the RAG system to retrieve the most relevant chunks for the query, which is then used by the generator LLM to generate accurate answers.

Q10. Can you give examples of real-world applications where RAG systems have demonstrated value?

AI search engines are a great example of how RAG systems have changed the way people find information online. AI search engines give you accurate, relevant answers by combining information retrieval with generative AI.

For instance, RAG-based AI search platforms like Perplexity AI improve the user experience by fetching the most recent and relevant information from large knowledge bases and then giving it back in the format that the user wants.

Q11. Explain the steps in the indexing process in a RAG pipeline.

There are four steps in the indexing process of a RAG pipeline: parsing, chunking, encoding, and storing. The parsing step deals with extracting the document content.

Then, the chunking step splits the extracted content into smaller pieces called chunks.

The encoding step uses an embedding model to convert chunks into dense numerical vectors called embeddings. Finally, these embeddings are saved in a vector database for efficient search and retrieval.

All these steps in the indexing process are performed offline.

Q12. Explain the importance of chunking in RAG.

Chunking in Retrieval-Augmented Generation (RAG) is crucial because it breaks down

large texts into smaller and semantically coherent segments called chunks. Proper chunking helps to find relevant information efficiently by creating focused chunks that maintain context and avoid irrelevant noise.

Choosing the right chunk size balances detail and context, optimizing both retrieval accuracy and computational efficiency. Ineffective chunking can lead to poor retrieval results and incoherent responses, which makes it a foundational step for successful RAG performance in real-world applications.

**DON'T
FORGET TO
LIKE,
COMMENT
AND
SHARE**

